

EVENTOS

INTRODUCCIÓN

- JavaScript permite realizar scripts con ambos métodos de programación: secuencial y basada en eventos.
- El nivel 1 de DOM no incluye especificaciones relativas a los eventos JavaScript. El nivel 2 de DOM incluye ciertos aspectos relacionados con los eventos y el nivel 3 de DOM incluye la especificación completa de los eventos de JavaScript. Desafortunadamente, la especificación de nivel 3 de DOM se publicó en el año 2004, más de diez años después de que los primeros navegadores incluyeran los eventos.
- Por este motivo, muchas de las propiedades y métodos actuales relacionados con los eventos son incompatibles con los de DOM. De hecho, navegadores como Internet Explorer tratan los eventos siguiendo su propio modelo incompatible con el estándar.

MODELO BÁSICO DE EVENTOS (I)

- El modelo simple de eventos se introdujo en la versión 4 del estándar HTML y se considera parte del nivel más básico de DOM. Aunque sus características son limitadas, es el único modelo que es compatible con todos los navegadores.
- Tipos de Eventos:

Evento	Descripción	Elementos para los que está definido
onblur	Deseleccionar el elemento	<button>, <input>, <label>, <select>, <textarea>, <body>
onchange	Deseleccionar un elemento que se ha modificado	<input>, <select>, <textarea>
onclick	Pinchar y soltar el ratón	Todos los elementos
ondblclick	Pinchar dos veces seguidas con el ratón	Todos los elementos
onfocus	Seleccionar un elemento	<button>, <input>, <label>, <select>, <textarea>, <body>
onkeydown	Pulsar una tecla y no soltarla	Elementos de formulario y <body>

MODELO BÁSICO DE EVENTOS (II)

Evento	Descripción	Elementos para los que está definido
onkeypress	Pulsar una tecla	Elementos de formulario y <body>
onkeyup	Soltar una tecla pulsada	Elementos de formulario y <body>
onload	Página cargada completamente	<body>
onmousedown	Pulsar un botón del ratón y no soltarlo	Todos los elementos
onmousemove	Mover el ratón	Todos los elementos
onmouseout	El ratón <i>"sale"</i> del elemento	<i>Todos los elementos</i>
onmouseover	El ratón <i>"entra"</i> en el elemento	<i>Todos los elementos</i>
onmouseup	Soltar el botón del ratón	Todos los elementos
onreset	Inicializar el formulario	<form>
onresize	Modificar el tamaño de la ventana	<body>
onselect	Seleccionar un texto	<input>, <textarea>
onsubmit	Enviar el formulario	<form>
onunload	Se abandona la página, por ejemplo al cerrar el navegador	<body>

ONCLICK

- Pinchar y soltar el ratón
- Si el manejador devuelve false se cancela la acción.

```
<script language="javascript">
function Aviso() {
    var mensaje="El contenido de la página podría herir la sensibilidad.\n"+
                "¿Seguro que deseas continuar?";
    return confirm (mensaje);
}
</script>
...
<a href="exámenes.html" onClick="return Aviso();">Examen online</a>
...
...
<a name="enlace1" href=http://www.san.gva.es> Web AVS</a>
...
document.links[0].onclick = new Function( "return confirm('Seguro que
    quieres visitar la página de Paco Tomás');" );
...
```

ONSUBMIT/ONRESET

- Ha sido pulsado el botón de submit /reset del formulario.
- Devolviendo false se anula la acción del botón.

```
<script language="javascript">
function ValidarDatos()
{
    var ok;

    ...
    if ( !ok ) return false;
    else return true;
}
</script>

...
<form name="miformulario" action="/dar_de_baja.php" method="POST"
    onSubmit="ValidarDatos();">

...
</form>
```

MANEJADORES DE EVENTOS: COMO ATRIBUTOS HTML

- La forma más sencilla de incluir un manejador de evento es mediante un atributo de HTML. El siguiente ejemplo muestra un mensaje cuando el usuario pincha en el botón:

```
<input type="button" value="Pinchame y verás"  
onclick="alert('Gracias por pinchar');" />
```

MANEJADORES DE EVENTOS Y VARIABLE THIS

- En los eventos de JavaScript, se puede utilizar la palabra reservada `this` para referirse al elemento HTML sobre el que se está ejecutando el evento.

```
<div id="elemento" style="padding: .2em; width: 150px; height: 60px; border: thin solid silver"
onmouseover = "document.getElementById('elemento').style.borderColor = 'black'"
onmouseout = "document.getElementById('elemento').style.borderColor = 'silver'">
Sección de contenidos...
</div>
```

```
<div style="padding: .2em; width: 150px; height: 60px; border: thin solid silver"
onmouseover="this.style.borderColor='black'"
onmouseout="this.style.borderColor='silver'">
Sección de contenidos...
</div>
```

MANEJADORES DE EVENTOS COMO FUNCIONES EXTERNAS

```
<input type="button" value="Pinchame y verás"
  onclick="alert('Gracias por pinchar');" />
```

- Se puede transformar en:

```
function muestraMensaje() {
  alert('Gracias por pinchar');
}
```

```
<input type="button" value="Pinchame y verás"
  onclick="muestraMensaje()" />
```

- En las funciones externas no es posible utilizar la variable `this` de la misma forma que en los manejadores insertados en los atributos HTML. Por tanto, es necesario pasar la variable `this` como parámetro a la función manejadora:

```
<div style="padding: .2em; width: 150px; height: 60px; border: thin solid
  silver"
```

```
onmouseover="resalta(this)" onmouseout="resalta(this)">
```

Sección de contenidos...

```
</div>
```

MANEJADORES DE EVENTOS SEMÁNTICOS

(I)

- Esta técnica consiste en asignar las funciones externas mediante las propiedades DOM de los elementos HTML.
- El código HTML resultante es muy "limpio", ya que no se mezcla con el código JavaScript.
- La técnica de los manejadores semánticos consiste en:
 1. Asignar un identificador único al elemento HTML mediante el atributo id.
 2. Crear una función de JavaScript encargada de manejar el evento.
 3. Asignar la función a un evento concreto del elemento HTML mediante DOM.
- Otra ventaja adicional de esta técnica es que las funciones externas pueden utilizar la variable `this` referida al elemento que origina el evento.

MANEJADORES DE EVENTOS SEMÁNTICOS

(II)

- El único inconveniente de este método es que los manejadores se asignan mediante las funciones DOM, que solamente se pueden utilizar después de que la página se ha cargado completamente. De esta forma, para que la asignación de los manejadores no resulte errónea, es necesario asegurarse de que la página ya se ha cargado.

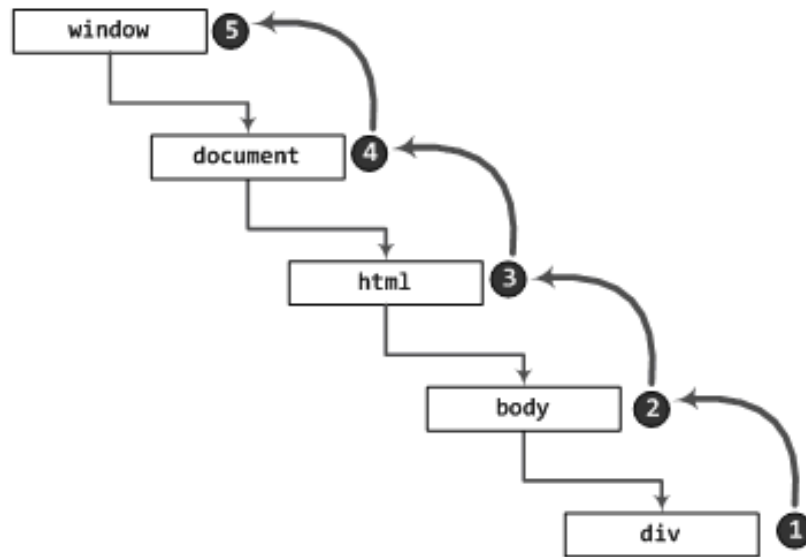
```
function muestraMensaje() {  
  alert('Gracias por pinchar');  
}
```

```
window.onload = function() {  
  document.getElementById("pinchable").onclick = muestraMensaje;  
}
```

```
<input id="pinchable" type="button" value="Pinchame y verás" />
```

EL FLUJO DE EVENTOS (I)

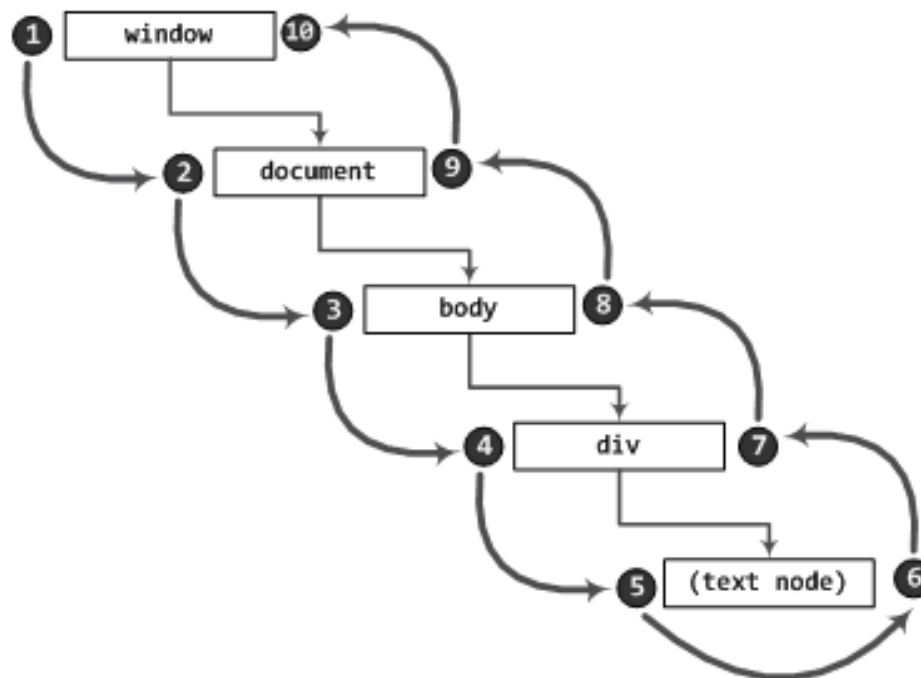
- El orden en el que se ejecutan los eventos asignados a cada elemento de la página es lo que constituye el flujo de eventos.
 - Event bubbling: en este modelo de flujo de eventos, el orden que se sigue es desde el elemento más específico hasta el elemento menos específico.
 - Event capturing: en ese otro modelo, el flujo de eventos se define desde el elemento menos específico hasta el elemento más específico.



Esquema de funcionamiento del “event bubbling”. Es el flujo de eventos que se incluye en Internet Explorer sin el objeto windows, en mozilla (firefox) si que se incluye este objeto windows.

EL FLUJO DE EVENTOS (II)

- El flujo de eventos definido en la especificación DOM soporta tanto el bubbling como el capturing, pero el "event capturing" se ejecuta en primer lugar. Los dos flujos de eventos recorren todos los objetos DOM desde el objeto document hasta el elemento más específico y viceversa. Además, la mayoría de navegadores que implementan los estándares, continúan el flujo hasta el objeto window.



MANEJADORES DE EVENTOS DE IE

- La familia de navegadores de Internet Explorer emplean los métodos `attachEvent()` y `detachEvent()` para asociar y desasociar manejadores de eventos. Todos los elementos y el objeto `window` disponen de estos métodos.

```
function muestraMensaje() {  
    alert("Has pulsado el ratón");  
}  
function muestraOtroMensaje() {  
    alert("Has pulsado el ratón y por eso se muestran estos mensajes");  
}  
var elDiv = document.getElementById("div_principal");  
elDiv.attachEvent("onclick", muestraMensaje);  
elDiv.attachEvent("onclick", muestraOtroMensaje);  
  
// Más adelante se decide desasociar la función del evento  
elDiv.detachEvent("onclick", muestraMensaje);
```

MANEJADORES DE EVENTOS DE DOM (I)

- La especificación DOM define otros dos métodos similares a los disponibles para Internet Explorer y denominados `addEventListener()` y `removeEventListener()` para asociar y desasociar manejadores de eventos.
- La principal diferencia entre estos métodos y los anteriores es que en este caso se requieren tres parámetros: el nombre del "event listener", una referencia a la función encargada de procesar el evento y el tipo de flujo de eventos al que se aplica.
- El primer argumento no es el nombre completo del evento como sucede en el modelo de Internet Explorer, sino que se debe eliminar el prefijo `on`. En otras palabras, si en Internet Explorer se utilizaba el nombre `onclick`, ahora se debe utilizar `click`.
- Si el tercer parámetro es `true`, el manejador se emplea en la fase de capture. Si el tercer parámetro es `false`, el manejador se asocia a la fase de bubbling.

MANEJADORES DE EVENTOS DE DOM (II)

```
function muestraMensaje() {  
    alert("Has pulsado el ratón");  
}  
function muestraOtroMensaje() {  
    alert("Has pulsado el ratón y por eso se muestran estos mensajes");  
}  
var elDiv = document.getElementById("div_principal");  
  
elDiv.addEventListener("click", muestraMensaje, true);  
elDiv.addEventListener("click", muestraOtroMensaje, true);
```

Si se asocia una función a un flujo de eventos determinado, esa función sólo se puede desasociar en el mismo tipo de flujo de eventos.

```
// Más adelante se decide desasociar la función al evento  
elDiv.removeEventListener("click", muestraMensaje, true);
```

EL OBJETO EVENT

- Cuando se produce un evento, no es suficiente con asignarle una función responsable de procesar ese evento. Normalmente, la función que procesa el evento necesita información relativa al evento producido: la tecla que se ha pulsado, la posición del ratón, el elemento que ha producido el evento, etc.
- El objeto event es el mecanismo definido por los navegadores para proporcionar toda esa información. Se trata de un objeto que se crea automáticamente cuando se produce un evento y que se destruye de forma automática cuando se han ejecutado todas las funciones asignadas al evento.

- Internet Explorer permite el acceso al objeto event a través del objeto window:

```
elDiv.onclick = function() { var elEvento = window.event; }
```

- El estándar DOM especifica que el objeto event es el único parámetro que se debe pasar a las funciones encargadas de procesar los eventos. Por tanto, en los navegadores que siguen los estándares, se puede acceder al objeto event a través del array de los argumentos de la función o es posible indicar el nombre del argumento de forma explícita:

```
elDiv.onclick = function() { var elEvento = arguments[0]; }
```

```
elDiv.onclick = function(elEvento) { .... }
```

PROPIEDADES DEL OBJETO EVENT

DEFINIDAS POR IE (I)

Propiedad/ Método	Devuelve	Descripción
altKey	Boolean	Devuelve true si se ha pulsado la tecla ALT y false en otro caso
button	Número entero	El botón del ratón que ha sido pulsado. Posibles valores: 0 – Ningún botón pulsado 1 – Se ha pulsado el botón izquierdo 2 – Se ha pulsado el botón derecho 3 – Se pulsan a la vez el botón izquierdo y el derecho 4 – Se ha pulsado el botón central 5 – Se pulsan a la vez el botón izquierdo y el central 6 – Se pulsan a la vez el botón derecho y el central 7 – Se pulsan a la vez los 3 botones Cancel
Bubble	Boolean	Si se establece un valor true, se detiene el flujo de eventos de tipo <i>bubbling</i>
clientX	Número entero	Coordenada X de la posición del ratón respecto del área visible de la ventana
clientY	Número entero	Coordenada Y de la posición del ratón respecto del área visible de la ventana
ctrlKey	Boolean	Devuelve true si se ha pulsado la tecla CTRL y false en otro caso
fromElement	Element	El elemento del que sale el ratón (para ciertos eventos de ratón)

PROPIEDADES DEL OBJETO EVENT

DEFINIDAS POR IE (II)

Propiedad/ Método	Devuelve	Descripción
keyCode	Número entero	En el evento keypress, indica el carácter de la tecla pulsada. En los eventos keydown y keyup indica el código numérico de la tecla pulsada
offsetX	Número entero	Coordenada X de la posición del ratón respecto del elemento que origina el evento
offsetY	Número entero	Coordenada Y de la posición del ratón respecto del elemento que origina el evento
repeat	Boolean	Devuelve true si se está produciendo el evento keydown de forma continuada y false en otro caso
returnValue	Boolean	Se emplea para cancelar la acción predefinida del evento
screenX	Número entero	Coordenada X de la posición del ratón respecto de la pantalla completa
screenY	Número entero	Coordenada Y de la posición del ratón respecto de la pantalla completa
shiftKey	Boolean	Devuelve true si se ha pulsado la tecla SHIFT y false en otro caso
srcElement	Element	El elemento que origina el evento
toElement	Element	El elemento al que entra el ratón (para ciertos eventos de ratón)

PROPIEDADES DEL OBJETO EVENT

DEFINIDAS POR IE (III)

Propiedad/ Método	Devuelve	Descripción
type	Cadena de texto	El nombre del evento
x	Número entero	Coordenada X de la posición del ratón respecto del elemento padre del elemento que origina el evento
y	Número entero	Coordenada Y de la posición del ratón respecto del elemento padre del elemento que origina el evento
srcElement	Element	El elemento que origina el evento
toElement	Element	El elemento al que entra el ratón (para ciertos eventos de ratón)
type	Cadena de texto	El nombre del evento
x	Número entero	Coordenada X de la posición del ratón respecto del elemento padre del elemento que origina el evento
y	Número entero	Coordenada Y de la posición del ratón respecto del elemento padre del elemento que origina el evento

Todas las propiedades salvo repeat son de lectura/escritura y por tanto, su valor se puede leer y/o establecer.

PROPIEDADES DEL OBJETO EVENT

DEFINIDAS POR DOM (I)

Propiedad/ Método	Devuelve	Descripción
altKey	Boolean	Devuelve true si se ha pulsado la tecla ALT y false en otro caso
bubbles	Boolean	Indica si el evento pertenece al flujo de eventos de <i>bubbling</i>
button	Número entero	El botón del ratón que ha sido pulsado. Posibles valores: 0 – Ningún botón pulsado 1 – Se ha pulsado el botón izquierdo 2 – Se ha pulsado el botón derecho 3 – Se pulsan a la vez el botón izquierdo y el derecho 4 – Se ha pulsado el botón central 5 – Se pulsan a la vez el botón izquierdo y el central 6 – Se pulsan a la vez el botón derecho y el central 7 – Se pulsan a la vez los 3 botones
cancelable	Boolean	Indica si el evento se puede cancelar
cancelBubble	Boolean	Indica si se ha detenido el flujo de eventos de tipo <i>bubbling</i>
charCode	Número entero	El código unicode del carácter correspondiente a la tecla pulsada
clientX	Número entero	Coordenada X de la posición del ratón respecto del área visible de la ventana
clientY	Número entero	Coordenada Y de la posición del ratón respecto del área visible de la ventana

PROPIEDADES DEL OBJETO EVENT

DEFINIDAS POR DOM(II)

Propiedad/ Método	Devuelve	Descripción
ctrlKey	Boolean	Devuelve true si se ha pulsado la tecla CTRL y false en otro caso
currentTarget	Element	El elemento que es el objetivo del evento
detail	Número entero	El número de veces que se han pulsado los botones del ratón
eventPhase	Número entero	La fase a la que pertenece el evento: 0 – Fase capturing 1 – En el elemento destino 2 – Fase bubbling
isChar	Boolean	Indica si la tecla pulsada corresponde a un carácter
keyCode	Número entero	Indica el código numérico de la tecla pulsada
metaKey	Número entero	Devuelve true si se ha pulsado la tecla META y false en otro caso
pageX	Número entero	Coordenada X de la posición del ratón respecto de la página
pageY	Número entero	Coordenada Y de la posición del ratón respecto de la página
preventDefault()	Función	Se emplea para cancelar la acción predefinida del evento

PROPIEDADES DEL OBJETO EVENT

DEFINIDAS POR DOM(III)

Propiedad/ Método	Devuelve	Descripción
relatedTarget	Element	El elemento que es el objetivo secundario del evento (relacionado con los eventos de ratón)
screenX	Número entero	Coordenada X de la posición del ratón respecto de la pantalla completa
screenY	Número entero	Coordenada Y de la posición del ratón respecto de la pantalla completa
shiftKey	Boolean	Devuelve true si se ha pulsado la tecla SHIFT y false en otro caso
stopPropagation()	Función	Se emplea para detener el flujo de eventos de tipo <i>bubbling</i>
target	Element	El elemento que origina el evento
timeStamp	Número	La fecha y hora en la que se ha producido el evento
type	Cadena de texto	El nombre del evento

Al contrario de lo que sucede con Internet Explorer, la mayoría de propiedades del objeto event de DOM son de sólo lectura. En concreto, solamente las siguientes propiedades son de lectura y escritura: altKey, button y keyCode.

EJEMPLOS (I)

- En ambos casos se utiliza la propiedad type para obtener el tipo de evento que se trata:

```
function procesaEvento(elEvento) {  
    if(elEvento.type == "click") {  
        alert("Has pulsado el raton");  
    }  
    else if(elEvento.type == "mouseover") {  
        alert("Has movido el raton");  
    }  
}  
elDiv.onclick = procesaEvento;  
elDiv.onmouseover = procesaEvento;
```

- Una de las principales diferencias es la forma en la que se obtiene el elemento que origina el evento.

```
// Internet Explorer  
var objetivo = elEvento.srcElement;  
// Navegadores que siguen los estándares  
var objetivo = elEvento.target;
```

EJEMPLOS (II)

- Otra diferencia importante es la relativa a la obtención del carácter correspondiente a la tecla pulsada. Cada tecla pulsada tiene asociados dos códigos diferentes: el primero es el código de la tecla que ha sido presionada (es un código de tecla interno para JavaScript) y el otro código es el que se refiere al carácter pulsado (código ASCII del carácter).

Propiedad KeyCode	Eventos de "pulsación de teclas" (onkeyup y onkeydown)	Eventos de "escribir con teclas" (onkeypress)
Internet Explorer	código interno	código ASCII del carácter pulsado
Navegadores que siguen los estándares	código interno	0

EJEMPLOS (III)

- En los eventos `onkeyup` y `onkeydown` se puede utilizar la misma propiedad en todos los navegadores:

```
function manejador(elEvento) {  
    var evento = elEvento || window.event;  
    alert("[ "+evento.type+" ] El código de la tecla pulsada es " +  
        evento.keyCode);  
}  
document.onkeyup = manejador;  
document.onkeydown = manejador;
```

- En el evento `onkeypress`:

```
function manejador(elEvento) {  
    var evento = elEvento || window.event;  
    //charCode obtiene el código ascii de la tecla pulsada, si no es IE  
    var codigo = evento.keyCode || evento.charCode;  
    //obtener el carácter del código ascii  
    var carácter = String.fromCharCode(codigo);  
}  
document.onkeypress = manejador;
```

TIPOS DE EVENTOS

- La lista completa de eventos que se pueden generar en un navegador se puede dividir en cuatro grandes grupos. La especificación de DOM define los siguientes grupos:
 - Eventos de ratón: se originan cuando el usuario emplea el ratón para realizar algunas acciones.
 - Eventos de teclado: se originan cuando el usuario pulsa sobre cualquier tecla de su teclado.
 - Eventos HTML: se originan cuando se producen cambios en la ventana del navegador o cuando se producen ciertas interacciones entre el cliente y el servidor.
 - Eventos DOM: se originan cuando se produce un cambio en la estructura DOM de la página. También se denominan "eventos de mutación".

EVENTOS DE RATÓN

- click: se produce cuando se pulsa el botón izquierdo del ratón. También se produce cuando el foco de la aplicación está situado en un botón y se pulsa la tecla ENTER.
- dblclick: se produce cuando se pulsa dos veces el botón izquierdo del ratón.
- mousedown: se produce cuando se pulsa cualquier botón del ratón.
- mouseout: se produce cuando el puntero del ratón se encuentra en el interior de un elemento y el usuario mueve el puntero a un lugar fuera de ese elemento.
- mouseover: se produce cuando el puntero del ratón se encuentra fuera de un elemento y el usuario mueve el puntero hacia un lugar en el interior del elemento.
- mouseup: se produce cuando se suelta cualquier botón del ratón que haya sido pulsado.
- mousemove: se produce (de forma continua) cuando el puntero del ratón se encuentra sobre un elemento.

PROPIEDADES DEL OBJETO EVENT PARA LOS EVENTOS DE RATÓN

- Las coordenadas del ratón (todas las coordenadas diferentes relativas a los distintos elementos)
- La propiedad `type`
- La propiedad `srcElement` (Internet Explorer) o `target` (DOM)
- Las propiedades `shiftKey`, `ctrlKey`, `altKey` y `metaKey` (sólo DOM)
- La propiedad `button` (sólo en los eventos `mousedown`, `mousemove`, `mouseout`, `mouseover` y `mouseup`)

Los eventos `mouseover` y `mouseout` tienen propiedades adicionales. Internet Explorer define la propiedad `fromElement`, que hace referencia al elemento desde el que el puntero del ratón se ha movido y `toElement` que es el elemento al que el puntero del ratón se ha movido. De esta forma, en el evento `mouseover`, la propiedad `toElement` es idéntica a `srcElement` y en el evento `mouseout`, la propiedad `fromElement` es idéntica a `srcElement`.

En los navegadores que soportan el estándar DOM, solamente existe una propiedad denominada `relatedTarget`. En el evento `mouseout`, `relatedTarget` apunta al elemento al que se ha movido el ratón. En el evento `mouseover`, `relatedTarget` apunta al elemento desde el que se ha movido el puntero del ratón.

EVENTOS Y PROPIEDADES DE TECLADO

- keydown: se produce cuando se pulsa cualquier tecla del teclado. También se produce de forma continua si se mantiene pulsada la tecla.
- keypress: se produce cuando se pulsa una tecla correspondiente a un carácter alfanumérico (no se tienen en cuenta teclas como SHIFT, ALT, etc.). También se produce de forma continua si se mantiene pulsada la tecla.
- keyup: se produce cuando se suelta cualquier tecla pulsada.

PROPIEDADES DEL OBJETO EVENT PARA LOS EVENTOS DE TECLADO

- La propiedad keyCode
- La propiedad charCode (sólo DOM)
- La propiedad srcElement (Internet Explorer) o target (DOM)
- Las propiedades shiftKey, ctrlKey, altKey y metaKey (sólo DOM)

EVENTOS HTML (I)

- load: se produce en el objeto window cuando la página se carga por completo. En el elemento `<img>` cuando se carga por completo la imagen. En el elemento `<object>` cuando se carga el objeto
- unload: se produce en el objeto window cuando la página desaparece por completo (al cerrar la ventana del navegador por ejemplo). En el elemento `<object>` cuando desaparece el objeto.
- abort: se produce en un elemento `<object>` cuando el usuario detiene la descarga del elemento antes de que haya terminado.
- error: se produce en el objeto window cuando se produce un error de JavaScript. En el elemento `<img>` cuando la imagen no se ha podido cargar por completo y en el elemento `<object>` cuando el elemento no se carga correctamente.
- select: se produce cuando se seleccionan varios caracteres de un cuadro de texto (`<input>` y `<textarea>`).
- change: se produce cuando un cuadro de texto (`<input>` y `<textarea>`) pierde el foco y su contenido ha variado. También se produce cuando varía el valor de un elemento `<select>`.

EVENTOS HTML (II)

- submit: se produce cuando se pulsa sobre un botón de tipo submit (`<input type="submit">`).
- reset: se produce cuando se pulsa sobre un botón de tipo reset (`<input type="reset">`).
- resize: se produce en el objeto window cuando se redimensiona la ventana del navegador.
- scroll: se produce en cualquier elemento que tenga una barra de scroll, cuando el usuario la utiliza. El elemento `<body>` contiene la barra de scroll de la página completa.
- focus: se produce en cualquier elemento (incluido el objeto window) cuando el elemento obtiene el foco.
- blur: se produce en cualquier elemento (incluido el objeto window) cuando el elemento pierde el foco.